Rapport

Atelier 2 : Procédures et fonctions stockées en MySQL

Base de données atelier_commerce hébergée sur Aiven (MySQL 8.4.11)

Nom et prénom : .....

Date : 11/11/2026

1. Objectif et environnement

Cet atelier consiste à créer et à utiliser des procédures et des fonctions stockées en MySQL : paramètres IN, OUT et INOUT, structures de contrôle (IF) et boucle WHILE. Tout le travail a été réalisé sur mon ordinateur, en ligne de commande.

- Serveur : service MySQL 8.4.11 hébergé sur Aiven (service atelie1module3), connexion SSL obligatoire.
- Base de données : atelier_commerce, la même que celle de l'atelier 1.
- Client : mysql.exe (MySQL Server 8.0) lancé depuis l'invite de commandes Windows.
- Tables créées pour cet atelier : stagiaires et facture. Les tables de l'atelier 1 n'ont pas été modifiées.

Remarque : les procédures et fonctions contiennent des « ; » à l'intérieur. On change donc temporairement le délimiteur avec DELIMITER / puis on le rétablit avec DELIMITER ;.

2. Connexion à Aiven et création de la table stagiaires

Connexion au serveur distant avec l'utilisateur avnadmin (le mot de passe est masqué à la saisie), vérification des bases disponibles, sélection de atelier_commerce, puis création de la table.

```
mysql -h <hôte Aiven> -P 28703 -u avnadmin -p --ssl-mode=REQUIRED defaultdb
SHOW DATABASES;
USE atelier_commerce;

CREATE TABLE stagiaires (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    ville VARCHAR(100)
);
```

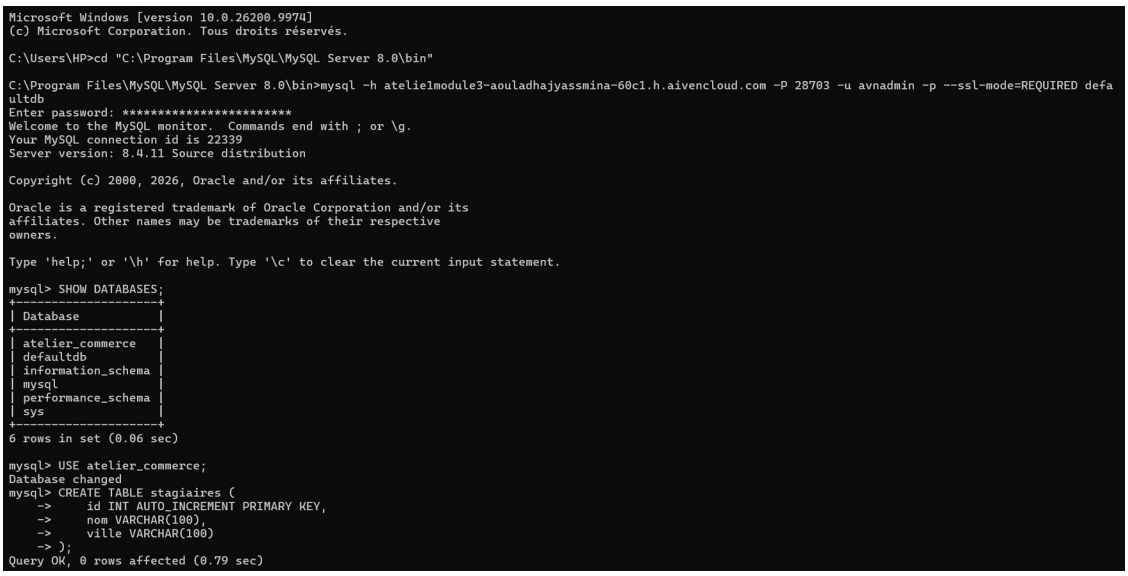


Figure 1 : Connexion à Aiven, liste des bases, sélection de atelier_commerce et création de la table stagiaires

3. Insertion des données et procédure afficher_stagiaires_par_ville

Insertion de cinq stagiaires, puis création d'une procédure à paramètre IN qui affiche les stagiaires d'une ville donnée. L'appel avec « Casablanca » retourne Ali et Khalid. (La ligne « DELIMITER ;CALL … » collée sur une seule ligne n'a pas été exécutée ; elle a été annulée avec Ctrl+C puis refaite correctement.)

```
INSERT INTO stagiaires (nom, ville) VALUES
('Ali','Casablanca'), ('Sara','Rabat'), ('Khalid','Casablanca'),
('Mouna','Marrakech'), ('Youssef','Rabat');

DELIMITER /
CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_param VARCHAR(100))
BEGIN
    SELECT * FROM stagiaires WHERE ville = ville_param;
END /
DELIMITER ;

CALL afficher_stagiaires_par_ville('Casablanca');
```

```
mysql> INSERT INTO stagiaires (nom, ville) VALUES
    -> ('Ali', 'Casablanca'),
    -> ('Sara', 'Rabat'),
    -> ('Khalid', 'Casablanca'),
    -> ('Mouna', 'Marrakech'),
    -> ('Youssef', 'Rabat');
Query OK, 5 rows affected (0.12 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM stagiaires;
+----+---------+------------+
| id | nom     | ville      |
+----+---------+------------+
|  1 | Ali     | Casablanca |
|  2 | Sara    | Rabat      |
|  3 | Khalid  | Casablanca |
|  4 | Mouna   | Marrakech  |
|  5 | Youssef | Rabat      |
+----+---------+------------+
5 rows in set (0.06 sec)

mysql> DELIMITER /
mysql> CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_param VARCHAR(100))
    -> BEGIN
    ->     SELECT * FROM stagiaires WHERE ville = ville_param;
    -> END /
Query OK, 0 rows affected (0.18 sec)

mysql> DELIMITER ;CALL afficher_stagiaires_par_ville('Casablanca');
mysql> ^C
mysql> DELIMITER ;
mysql> CALL afficher_stagiaires_par_ville('Casablanca');
+----+--------+------------+
| id | nom    | ville      |
+----+--------+------------+
|  1 | Ali    | Casablanca |
|  3 | Khalid | Casablanca |
+----+--------+------------+
2 rows in set (0.30 sec)
```

Figure 2 : Insertion des stagiaires, création et appel de la procédure afficher_stagiaires_par_ville

4. Procédure compter_stagiaires et fonction nombre_stagiaires_par_ville

La procédure compter_stagiaires calcule le nombre total de stagiaires dans une variable locale puis l'affiche (résultat : 5). La fonction nombre_stagiaires_par_ville retourne un entier : 2 pour Rabat et 2 pour Casablanca.

```
DELIMITER /
CREATE PROCEDURE compter_stagiaires()
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total FROM stagiaires;
    SELECT total AS 'Nombre de stagiaires';
END /

CREATE FUNCTION nombre_stagiaires_par_ville(ville_param VARCHAR(100))
RETURNS INT
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total FROM stagiaires WHERE ville = ville_param;
    RETURN total;
END /
DELIMITER ;
```

```
CALL compter_stagiaires();
SELECT nombre_stagiaires_par_ville('Rabat');
```

```
Query OK, 0 rows affected (0.31 sec)

mysql> DELIMITER /
mysql> CREATE PROCEDURE compter_stagiaires()
    -> BEGIN
    ->     DECLARE total INT;
    ->     SELECT COUNT(*) INTO total FROM stagiaires;
    ->     SELECT total AS 'Nombre de stagiaires';
    -> END /
Query OK, 0 rows affected (0.18 sec)

mysql> DELIMITER ;
mysql> CALL compter_stagiaires();
+----------------------+
| Nombre de stagiaires |
+----------------------+
|                    5 |
+----------------------+
1 row in set (0.17 sec)

Query OK, 0 rows affected (0.17 sec)

mysql> DELIMITER /
mysql> CREATE FUNCTION nombre_stagiaires_par_ville(ville_param VARCHAR(100))
    -> RETURNS INT
    -> BEGIN
    ->     DECLARE total INT;
    ->     SELECT COUNT(*) INTO total FROM stagiaires WHERE ville = ville_param;
    ->     RETURN total;
    -> END /
Query OK, 0 rows affected (0.19 sec)

mysql> DELIMITER ;
mysql> SELECT nombre_stagiaires_par_ville('Rabat');
+--------------------------------------+
| nombre_stagiaires_par_ville('Rabat') |
+--------------------------------------+
|                                    2 |
+--------------------------------------+
1 row in set (0.15 sec)
```

Figure 3 : Création de compter_stagiaires et de la fonction nombre_stagiaires_par_ville, test avec Rabat

```
mysql> SELECT nombre_stagiaires_par_ville('Casablanca');
+-------------------------------------------+
| nombre_stagiaires_par_ville('Casablanca') |
+-------------------------------------------+
|                                         2 |
+-------------------------------------------+
1 row in set (0.06 sec)

mysql> CALL compter_stagiaires();
+----------------------+
| Nombre de stagiaires |
+----------------------+
|                    5 |
+----------------------+
1 row in set (0.17 sec)

Query OK, 0 rows affected (0.18 sec)

mysql> SHOW PROCEDURE STATUS WHERE Db = DATABASE()\G
*************************** 1. row ***************************
                  Db: atelier_commerce
                Name: afficher_stagiaires_par_ville
                Type: PROCEDURE
            Language: SQL
             Definer: avnadmin@%
            Modified: 2026-11-11 12:05:19
             Created: 2026-11-11 12:05:19
       Security_type: DEFINER
             Comment:
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
  Database Collation: utf8mb4_0900_ai_ci
*************************** 2. row ***************************
                  Db: atelier_commerce
                Name: compter_stagiaires
                Type: PROCEDURE
            Language: SQL
             Definer: avnadmin@%
            Modified: 2026-11-11 12:06:32
             Created: 2026-11-11 12:06:32
```

Figure 4 : Tests de la fonction (Casablanca) et de la procédure, puis début de SHOW PROCEDURE STATUS

5. Liste des procédures, des fonctions et des tables

```
SHOW PROCEDURE STATUS WHERE Db = DATABASE()\G
SHOW FUNCTION STATUS WHERE Db = DATABASE()\G
SHOW TABLES;
```

Les deux procédures et la fonction créées apparaissent bien dans la base atelier_commerce. SHOW TABLES confirme la présence de stagiaires à côté des tables de l'atelier 1.

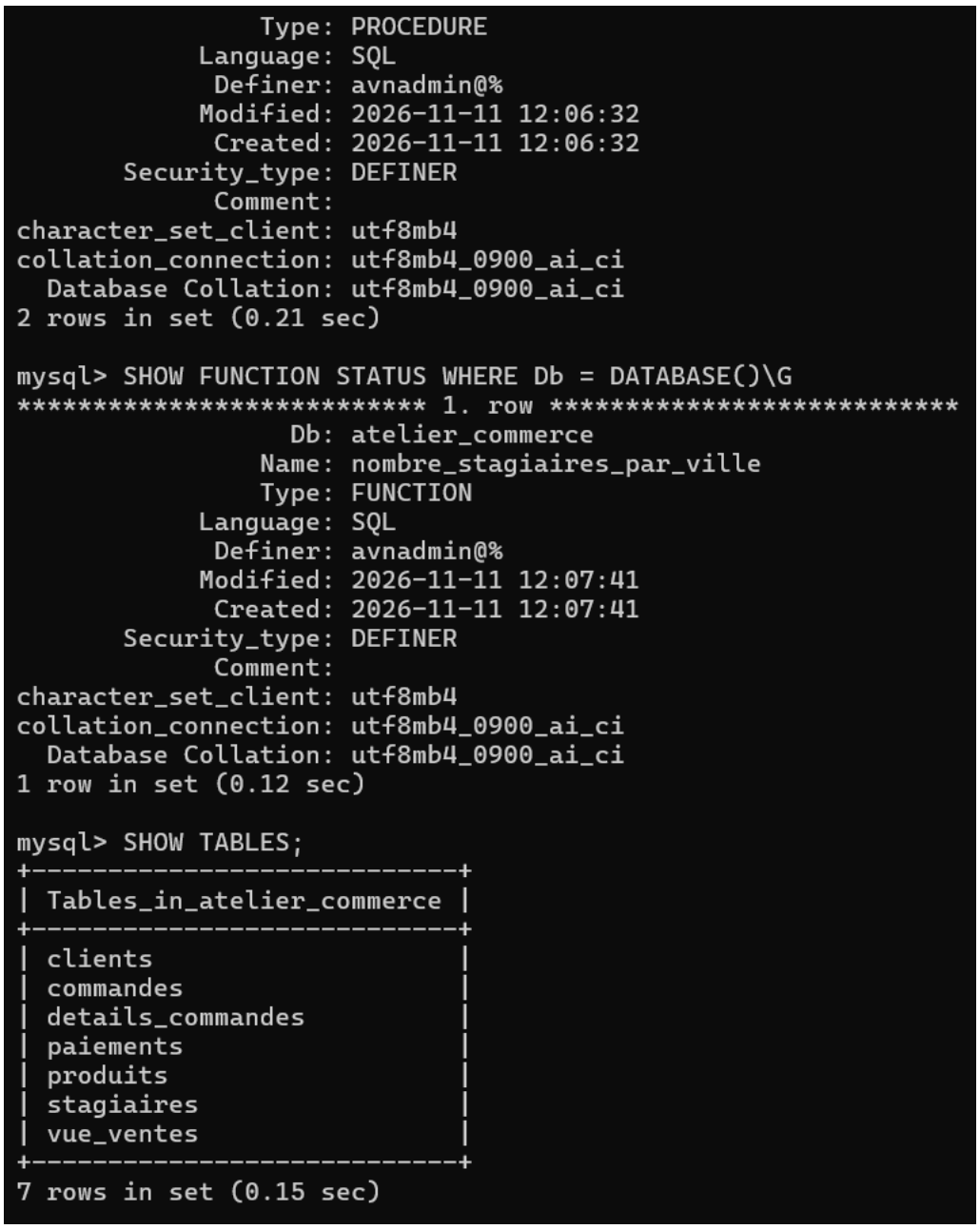


Figure 5 : Fin de SHOW PROCEDURE STATUS, SHOW FUNCTION STATUS et SHOW TABLES

6. Exemple pratique : facturation de l'eau

Une boucle WHILE insère 100 factures avec des quantités croissantes (Client1 avec 1 m³ jusqu'à Client100 avec 100 m³). La table contient bien 100 lignes après un seul appel de la procédure.

```
CREATE TABLE facture (
    id_facture INT AUTO_INCREMENT PRIMARY KEY,
    nom_client VARCHAR(255),
    qte_eau INT
);

DELIMITER /
CREATE PROCEDURE inserer_factures()
BEGIN
    DECLARE compteur INT DEFAULT 1;
```

```
    WHILE compteur <= 100 DO
        INSERT INTO facture (nom_client, qte_eau)
        VALUES (CONCAT('Client', compteur), compteur);
        SET compteur = compteur + 1;
    END WHILE;
END /
DELIMITER ;

CALL inserer_factures();
SELECT COUNT(*) FROM facture;
```

```
7 rows in set (0.15 sec)

mysql> ^C
mysql> CREATE TABLE facture (
    ->     id_facture INT AUTO_INCREMENT PRIMARY KEY,
    ->     nom_client VARCHAR(255),
    ->     qte_eau INT
    -> );
Query OK, 0 rows affected (0.18 sec)

mysql> DELIMITER /
mysql> CREATE PROCEDURE inserer_factures()
    -> BEGIN
    ->     DECLARE compteur INT DEFAULT 1;
    ->     WHILE compteur <= 100 DO
    ->         INSERT INTO facture (nom_client, qte_eau)
    ->         VALUES (CONCAT('Client', compteur), compteur);
    ->         SET compteur = compteur + 1;
    ->     END WHILE;
    -> END /
Query OK, 0 rows affected (0.17 sec)

mysql> DELIMITER ;
mysql> CALL inserer_factures();
Query OK, 1 row affected (0.24 sec)

mysql> SELECT COUNT(*) FROM facture;
+----------+
| COUNT(*) |
+----------+
|      100 |
+----------+
1 row in set (0.13 sec)

mysql> DELIMITER /
mysql> CREATE FUNCTION calculer_montant(qte INT)
    -> RETURNS DECIMAL(10,2)
    -> BEGIN
    ->     DECLARE montant DECIMAL(10,2);
    ->     IF qte <= 10 THEN
```

Figure 6 : Création de la table facture, de la procédure inserer_factures, appel et comptage (100 lignes), début de calculer_montant

La fonction calculer_montant applique le barème par tranches avec IF / ELSEIF / ELSE : 1 € par m³ jusqu'à 10 m³, 1,5 € par m³ de 11 à 30 m³, 2 € par m³ au-delà de 30 m³.

```
DELIMITER /
CREATE FUNCTION calculer_montant(qte INT)
```

```
RETURNS DECIMAL(10,2)
BEGIN
    DECLARE montant DECIMAL(10,2);
    IF qte <= 10 THEN
        SET montant = qte * 1;
    ELSEIF qte <= 30 THEN
        SET montant = (10 * 1) + (qte - 10) * 1.5;
    ELSE
        SET montant = (10 * 1) + (20 * 1.5) + (qte - 30) * 2;
    END IF;
    RETURN montant;
END /
DELIMITER ;

SELECT id_facture, nom_client, qte_eau,
       calculer_montant(qte_eau) AS montant_a_payer
FROM facture;
```

```
    ->     IF qte <= 10 THEN
    ->         SET montant = qte * 1;
    ->     ELSEIF qte <= 30 THEN
    ->         SET montant = (10 * 1) + (qte - 10) * 1.5;
    ->     ELSE
    ->         SET montant = (10 * 1) + (20 * 1.5) + (qte - 30) * 2;
    ->     END IF;
    ->     RETURN montant;
    -> END /
Query OK, 0 rows affected (0.10 sec)

mysql> DELIMITER ;
mysql> SELECT id_facture, nom_client, qte_eau,
    ->        calculer_montant(qte_eau) AS montant_a_payer
    -> FROM facture;
+------------+------------+---------+-----------------+
| id_facture | nom_client | qte_eau | montant_a_payer |
+------------+------------+---------+-----------------+
|          1 | Client1    |       1 |            1.00 |
|          2 | Client2    |       2 |            2.00 |
|          3 | Client3    |       3 |            3.00 |
|          4 | Client4    |       4 |            4.00 |
|          5 | Client5    |       5 |            5.00 |
|          6 | Client6    |       6 |            6.00 |
|          7 | Client7    |       7 |            7.00 |
|          8 | Client8    |       8 |            8.00 |
|          9 | Client9    |       9 |            9.00 |
|         10 | Client10   |      10 |           10.00 |
|         11 | Client11   |      11 |           11.50 |
|         12 | Client12   |      12 |           13.00 |
|         13 | Client13   |      13 |           14.50 |
|         14 | Client14   |      14 |           16.00 |
|         15 | Client15   |      15 |           17.50 |
|         16 | Client16   |      16 |           19.00 |
|         17 | Client17   |      17 |           20.50 |
|         18 | Client18   |      18 |           22.00 |
|         19 | Client19   |      19 |           23.50 |
|         20 | Client20   |      20 |           25.00 |
|         21 | Client21   |      21 |           26.50 |
|         22 | Client22   |      22 |           28.00 |
|         23 | Client23   |      23 |           29.50 |
```

Figure 7 : Fin de la fonction calculer_montant et début du résultat sur les 100 factures

Vérification sur des valeurs choisies : 5 m³ donne 5,00 € ; 10 m³ donne 10,00 € ; 20 m³ donne 25,00 € ; 30 m³ donne 40,00 € ; 50 m³ donne 80,00 € ; 100 m³ donne 180,00 €. Ces résultats correspondent au calcul à la main (par exemple 20 m³ = 10×1 + 10×1,5 = 25).

```
|         98 | Client98   |      98 |          176.00 |
|         99 | Client99   |      99 |          178.00 |
|        100 | Client100  |     100 |          180.00 |
+------------+------------+---------+-----------------+
100 rows in set (0.17 sec)

mysql> SELECT id_facture, nom_client, qte_eau,
    ->        calculer_montant(qte_eau) AS montant_a_payer
    -> FROM facture
    -> WHERE qte_eau IN (5, 10, 20, 30, 50);
+------------+------------+---------+-----------------+
| id_facture | nom_client | qte_eau | montant_a_payer |
+------------+------------+---------+-----------------+
|          5 | Client5    |       5 |            5.00 |
|         10 | Client10   |      10 |           10.00 |
|         20 | Client20   |      20 |           25.00 |
|         30 | Client30   |      30 |           40.00 |
|         50 | Client50   |      50 |           80.00 |
+------------+------------+---------+-----------------+
5 rows in set (0.11 sec)

mysql> INSERT INTO stagiaires (nom, ville) VALUES ('Test', 'Fes');
Query OK, 1 row affected (0.06 sec)

mysql> SELECT * FROM stagiaires;
+----+---------+------------+
| id | nom     | ville      |
+----+---------+------------+
|  1 | Ali     | Casablanca |
|  2 | Sara    | Rabat      |
|  3 | Khalid  | Casablanca |
|  4 | Mouna   | Marrakech  |
|  5 | Youssef | Rabat      |
|  6 | Test    | Fes        |
+----+---------+------------+
6 rows in set (0.14 sec)

mysql> DELIMITER /
mysql> CREATE PROCEDURE supprimer_stagiaire(IN p_id INT)
    -> BEGIN
    ->     DELETE FROM stagiaires WHERE id = p_id;
```

Figure 8 : Fin du résultat (Client100 = 180,00 €) et contrôle sur les quantités 5, 10, 20, 30 et 50

7. Paramètres IN, OUT et INOUT

Paramètre IN : la procédure supprimer_stagiaire reçoit l'identifiant à supprimer. Pour ne pas perdre de données, un stagiaire de test (id 6) a été ajouté puis supprimé ; il reste ensuite les 5 stagiaires d'origine.

```
INSERT INTO stagiaires (nom, ville) VALUES ('Test', 'Fes');

DELIMITER /
CREATE PROCEDURE supprimer_stagiaire(IN p_id INT)
BEGIN
    DELETE FROM stagiaires WHERE id = p_id;
```

```
END /
DELIMITER ;

CALL supprimer_stagiaire(6);
```

Paramètre OUT : la procédure compter_stagiaires_out renvoie le nombre de stagiaires dans la variable utilisateur @nb (résultat : 5).

```
DELIMITER /
CREATE PROCEDURE compter_stagiaires_out(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM stagiaires;
END /
DELIMITER ;

CALL compter_stagiaires_out(@nb);
SELECT @nb AS total_stagiaires;
```

```
    -> END /
Query OK, 0 rows affected (0.06 sec)

mysql> DELIMITER ;
mysql>
mysql> CALL supprimer_stagiaire(6);
Query OK, 1 row affected (0.06 sec)

mysql> SELECT * FROM stagiaires;
+----+---------+------------+
| id | nom     | ville      |
+----+---------+------------+
|  1 | Ali     | Casablanca |
|  2 | Sara    | Rabat      |
|  3 | Khalid  | Casablanca |
|  4 | Mouna   | Marrakech  |
|  5 | Youssef | Rabat      |
+----+---------+------------+
5 rows in set (0.06 sec)

mysql> DELIMITER /
mysql> CREATE PROCEDURE compter_stagiaires_out(OUT total INT)
    -> BEGIN
    ->     SELECT COUNT(*) INTO total FROM stagiaires;
    -> END /
Query OK, 0 rows affected (0.06 sec)

mysql> DELIMITER ;
mysql>
mysql> CALL compter_stagiaires_out(@nb);
Query OK, 1 row affected (0.08 sec)

mysql> SELECT @nb AS total_stagiaires;
+------------------+
| total_stagiaires |
+------------------+
|                5 |
+------------------+
1 row in set (0.21 sec)

mysql> DESCRIBE produits;
```

Figure 9 : Suppression du stagiaire de test avec le paramètre IN, puis procédure à paramètre OUT (@nb = 5)

Paramètre INOUT : la table produits de l'atelier 1 ne contient pas de colonne stock (DESCRIBE produits). Pour ne pas la modifier, une table séparée produits_stock a été créée uniquement pour tester l'exemple du cours. Le stock passe de 50 à 60 : la valeur 10 entre dans la procédure et la variable ressort avec la valeur 60.

```
CREATE TABLE produits_stock (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50),
    stock INT
);
INSERT INTO produits_stock (nom, stock) VALUES ('Stylo', 50);

DELIMITER /
CREATE PROCEDURE augmenter_stock(INOUT qte INT, IN p_id INT)
BEGIN
    UPDATE produits_stock SET stock = stock + qte WHERE id = p_id;
    SELECT stock INTO qte FROM produits_stock WHERE id = p_id;
```

```
END /
DELIMITER ;

SET @ajout = 10;
CALL augmenter_stock(@ajout, 1);
SELECT @ajout AS nouveau_stock;
```

```
+---------------+---------------+------+-----+---------+----------------+
| Field         | Type          | Null | Key | Default | Extra          |
+---------------+---------------+------+-----+---------+----------------+
| id            | int           | NO   | PRI | NULL    | auto_increment |
| nom_produit   | varchar(50)   | YES  | MUL | NULL    |                |
| prix_unitaire | decimal(10,2) | YES  |     | NULL    |                |
+---------------+---------------+------+-----+---------+----------------+
3 rows in set (0.17 sec)

mysql> CREATE TABLE produits_stock (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     nom VARCHAR(50),
    ->     stock INT
    -> );
Query OK, 0 rows affected (0.16 sec)

mysql>
mysql> INSERT INTO produits_stock (nom, stock) VALUES ('Stylo', 50);
Query OK, 1 row affected (0.11 sec)

mysql>
mysql> SELECT * FROM produits_stock;
+----+-------+-------+
| id | nom   | stock |
+----+-------+-------+
|  1 | Stylo |    50 |
+----+-------+-------+
1 row in set (0.17 sec)

mysql> DELIMITER /
mysql> CREATE PROCEDURE augmenter_stock(INOUT qte INT, IN p_id INT)
    -> BEGIN
    ->     UPDATE produits_stock SET stock = stock + qte WHERE id = p_id;
    ->     SELECT stock INTO qte FROM produits_stock WHERE id = p_id;
    -> END /
Query OK, 0 rows affected (0.15 sec)

mysql> DELIMITER ;
mysql> SET @ajout = 10;
Query OK, 0 rows affected (0.17 sec)
```

Figure 10 : DESCRIBE produits, création de produits_stock et de la procédure à paramètre INOUT

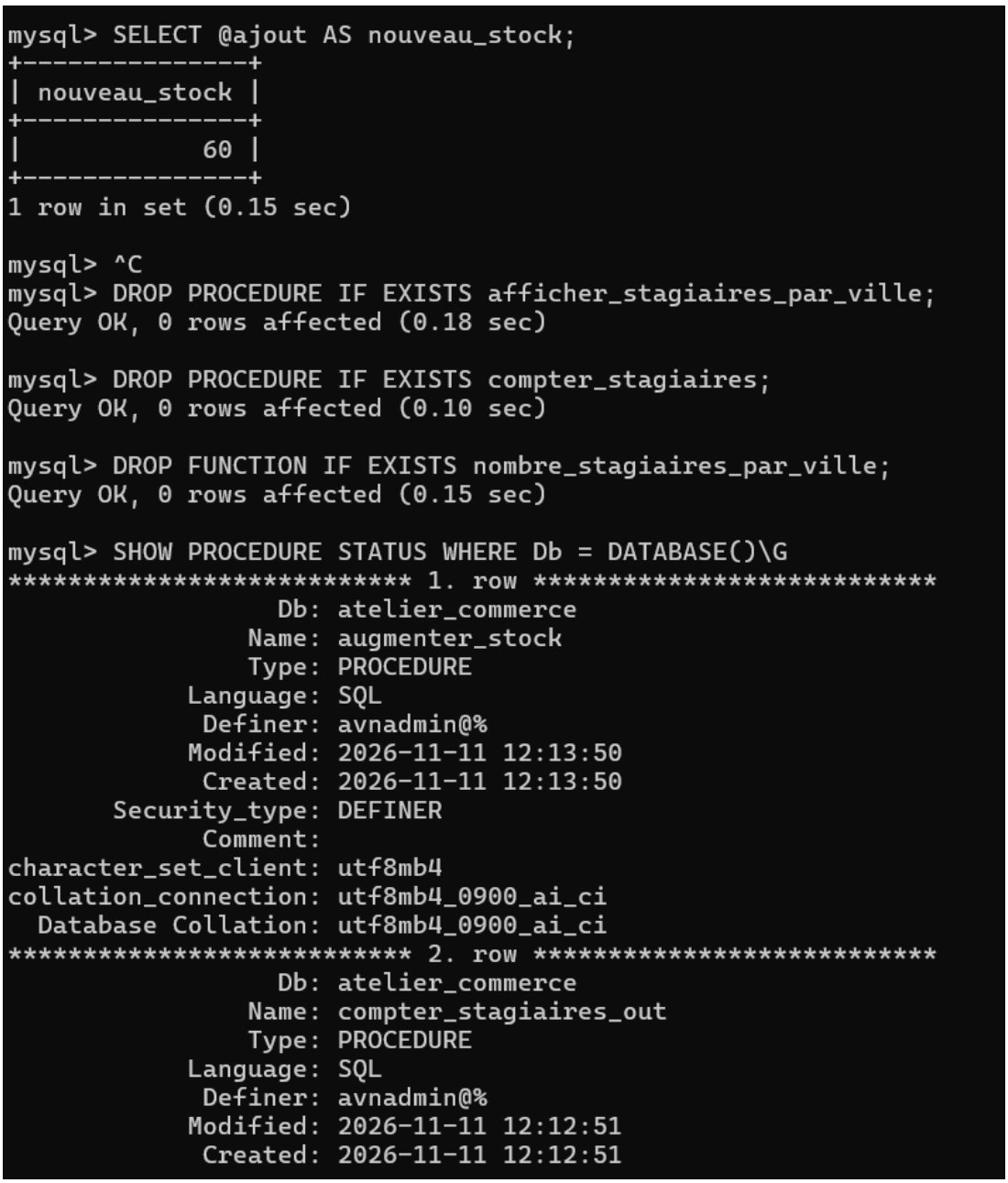


Figure 11 : Résultat de l'INOUT (nouveau_stock = 60), suppression des procédures et fonction, début de SHOW PROCEDURE STATUS

8. Suppression des procédures et fonctions

Comme demandé dans le cours, les deux procédures et la fonction de l'étude de cas ont été supprimées. Les vérifications SHOW montrent qu'il reste uniquement les objets utilisés dans les autres parties : augmenter_stock, compter_stagiaires_out, inserer_factures et supprimer_stagiaire (procédures), ainsi que calculer_montant (fonction). La table temporaire produits_stock a ensuite été supprimée.

```
DROP PROCEDURE IF EXISTS afficher_stagiaires_par_ville;
DROP PROCEDURE IF EXISTS compter_stagiaires;
DROP FUNCTION IF EXISTS nombre_stagiaires_par_ville;

SHOW PROCEDURE STATUS WHERE Db = DATABASE()\G
SHOW FUNCTION STATUS WHERE Db = DATABASE()\G
```

```
DROP TABLE produits_stock;
```

```
            Comment:
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
  Database Collation: utf8mb4_0900_ai_ci
*************************** 3. row ***************************
                Db: atelier_commerce
              Name: inserer_factures
              Type: PROCEDURE
          Language: SQL
           Definer: avnadmin@%
          Modified: 2026-11-11 12:10:13
           Created: 2026-11-11 12:10:13
      Security_type: DEFINER
            Comment:
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
  Database Collation: utf8mb4_0900_ai_ci
*************************** 4. row ***************************
                Db: atelier_commerce
              Name: supprimer_stagiaire
              Type: PROCEDURE
          Language: SQL
           Definer: avnadmin@%
          Modified: 2026-11-11 12:12:38
           Created: 2026-11-11 12:12:38
      Security_type: DEFINER
            Comment:
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
  Database Collation: utf8mb4_0900_ai_ci
4 rows in set (0.14 sec)

mysql> SHOW FUNCTION STATUS WHERE Db = DATABASE()\G
*************************** 1. row ***************************
                Db: atelier_commerce
              Name: calculer_montant
              Type: FUNCTION
          Language: SQL
           Definer: avnadmin@%
          Modified: 2026-11-11 12:11:15
           Created: 2026-11-11 12:11:15
```

Figure 12 : Suite de SHOW PROCEDURE STATUS (4 procédures restantes) et SHOW FUNCTION STATUS (calculer_montant)

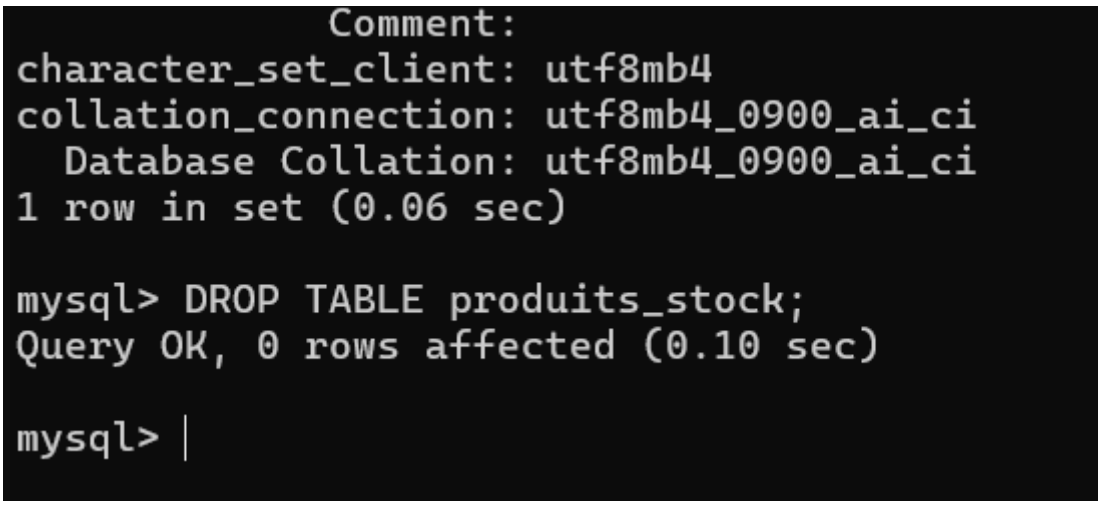


Figure 13 : Suppression de la table de test produits_stock

9. Conclusion

Cet atelier m'a permis de créer des procédures et des fonctions stockées, d'utiliser les paramètres IN, OUT et INOUT, les variables, la condition IF et la boucle WHILE, puis de les lister et de les supprimer. Toutes les commandes ont été exécutées avec succès sur le serveur MySQL distant Aiven depuis mon ordinateur.